Unit 3

Project Monitoring

3.1 Experiment 1: Project Performance Analysis

3.1.1 Cost, Time, Scope and Performance Monitoring

Project performance monitoring involves tracking actual progress against planned baselines across three key dimensions: cost, time, and scope [2]. For the Sanskrit NLP Framework project, Earned Value Management (EVM) is used as the primary performance monitoring technique.

3.1.2 Key Performance Indicators (KPIs)

The following KPIs have been established:

Table 3.1: Key Performance Indicators

KPI	**Target**	**Measurement Method**
Schedule Variance (SV)	$> -5\%$	EVMS – Compare EV vs PV
Cost Variance (CV)	$> -5\%$	EVMS – Compare EV vs AC
Schedule Performance Index (SPI)	> 0.95	EV / PV
Cost Performance Index (CPI)	> 0.95	EV / AC
Defect Density	< 2 per KLOC	Code review + testing
Requirement Stability Index (RSI)	$> 90\%$	(Initial requirements – Changes) / Initial
Linguist Validation Accuracy	$> 92\%$	Expert review of outputs
Customer Satisfaction (Post-UAT)	$> 85\%$	Survey of 10 beta users

3.1.3 Earned Value Analysis (EVA) – Month 6 Status

Reporting Period: End of Month 6 (after completion of Activity C – Panini rule formalization)

Table 3.2: Earned Value Analysis Parameters

Parameter	**Symbol**	**Value (INR Lakhs)**
Budget at Completion	BAC	37.00
Planned Value (cumulative)	PV	18.00
Earned Value (cumulative)	EV	16.50
Actual Cost (cumulative)	AC	19.00

3.1.4 Variance Analysis

$$\text{Cost Variance (CV)} = EV - AC = 16.50 - 19.00 = -2.50 \text{ Lakhs (Over budget)}$$

$$\text{Schedule Variance (SV)} = EV - PV = 16.50 - 18.00 = -1.50 \text{ Lakhs (Behind schedule)}$$

3.1.5 Cost and Schedule Performance Indices

$$\text{Cost Performance Index (CPI)} = \frac{EV}{AC} = \frac{16.50}{19.00} = 0.87$$

$$\text{Schedule Performance Index (SPI)} = \frac{EV}{PV} = \frac{16.50}{18.00} = 0.92$$

3.1.6 Forecasting

$$\text{Estimate at Completion (EAC)} = \frac{BAC}{CPI} = \frac{37.00}{0.87} = 42.53 \text{ Lakhs}$$

$$\text{Estimate to Complete (ETC)} = EAC - AC = 42.53 - 19.00 = 23.53 \text{ Lakhs}$$

$$\text{To-Complete Performance Index (TCPI)} = \frac{BAC - EV}{BAC - AC} = \frac{37.00 - 16.50}{37.00 - 19.00} = \frac{20.50}{18.00} = 1.14$$

3.1.7 Performance Improvement Plan

Based on the Month 6 analysis (CPI = 0.87, SPI = 0.92), the following corrective actions are recommended:

Table 3.3: Performance Improvement Plan

Action	Expected Impact	Cost
Add one junior NLP engineer to remaining activities	Increase SPI to 0.97	INR 60,000/month
Accelerate linguist review cycles (parallel reviews)	Reduce schedule slip by 5 days	INR 0 (process change)
Re-allocate contingency (INR 2 Lakhs) for critical path	Buffer against further delays	INR 2 Lakhs
Weekly EVA reporting for real-time tracking	Early detection of variances	INR 0

3.1.8 Project Controlling Techniques

Cost Controlling Techniques

- **Earned Value Management (EVM)**: Monthly tracking of CV and CPI
- **Budget Monitoring**: Weekly review of actual vs planned expenditure
- **Contingency Reserve Management**: Track reserve usage against risk triggers
- **Cost Baseline Control**: Formal change control for any budget adjustment

Schedule Controlling Techniques

- **Critical Path Monitoring**: Weekly tracking of critical path activities
- **Slack Management**: Monitor slack on non-critical paths
- **Milestone Tracking**: Dashboard of key milestone completion
- **Resource Loading**: Daily tracking of resource allocation

Scope Controlling Techniques

- **Requirements Traceability Matrix**: Link each requirement to deliverables
- **Change Control Board**: Formal review of all scope change requests

- **Scope Verification**: Customer acceptance at each milestone

3.1.9 Project Change Control

Table 3.4: Change Control Process

Step	Description
1. Change Request	Formal submission using Change Request Form
2. Impact Analysis	Assess impact on cost, schedule, scope, quality
3. CCB Review	Change Control Board meets weekly
4. Decision	Approve, reject, or defer with conditions
5. Implementation	Update project documents and communicate
6. Verification	Verify change was implemented correctly

Change Control Board (CCB) Members:

- Project Manager (Chair)
- Lead Sanskrit Linguist
- Lead NLP Engineer
- Quality Assurance Lead
- Sponsor Representative (for major changes)

3.1.10 Project Quality Controlling Techniques

Table 3.5: Quality Control Techniques

Technique	Application	Frequency
Code Reviews	Peer review of all code changes	Daily
Unit Testing	Automated pytest suite	Per commit
Integration Testing	API endpoint validation	Per sprint
Linguist Validation	Expert review of Panini rules	Weekly
Defect Tracking	JIRA for bug tracking	Continuous
Statistical Sampling	Random validation of outputs	Monthly

3.1.11 Revised Forecast After Corrective Actions

Table 3.6: Revised Forecast

Parameter	Before Correction	After Correction
EAC (INR Lakhs)	42.53	39.80
Projected Completion Delay	18 days	8 days
CPI (forward-looking)	0.87	0.94
SPI (forward-looking)	0.92	0.97

3.2 Experiment 2: Risk Management and Mitigation

3.2.1 Project Risk Analysis

Risk analysis involves identifying, assessing, and prioritizing risks that could affect project objectives [2]. Six key risks were identified through brainstorming sessions:

Table 3.7: Risk Identification and Assessment

Risk ID	Risk Description	Probability (P)	Impact (I)	Risk Score (P × I)
R01	Shortage of Sanskrit linguists with NLP knowledge	High (0.7)	High (0.8)	0.56
R02	Panini rule formalization takes longer than expected	Medium (0.5)	High (0.8)	0.40
R03	Low accuracy (<85%) on complex sandhi phenomena	Medium (0.5)	High (0.7)	0.35
R04	Cloud cost overrun due to large corpus processing	High (0.6)	Medium (0.5)	0.30
R05	Low adoption by academic institutions	Low (0.3)	High (0.7)	0.21
R06	Dependency on third-party Sanskrit corpus licenses	Medium (0.4)	Medium (0.5)	0.20

3.2.2 Risk Matrix (Probability-Impact Grid)

```
                    Low Impact    Medium Impact    High Impact
High Probability        -             R04              R01
Medium Probability      -             R06            R02, R03
Low Probability         -              -               R05
```

3.2.3 Risk Response Strategies

Table 3.8: Risk Response Strategies

Risk ID	Response Strategy	Specific Actions	Owner
R01	Mitigate	Partner with 2 Sanskrit universities for part-time linguists; Budget INR 3 Lakhs for honorarium	Project Manager
R02	Mitigate	Develop incremental rule engine; Release v1 with 500 sutras, v2 with 1000 sutras	NLP Lead
R03	Mitigate	Build gold-standard test dataset of 5000 words with expert validation before coding begins	QA Lead
R04	Mitigate	Use spot instances for non-critical batch processing; Set up budget alerts at INR 3 Lakhs	Cloud Engineer
R05	Accept + Contingency	Offer free 2-year pilot to top 5 institutions; Build case studies	Product Manager
R06	Transfer	Use open-source Sanskrit corpora (DCS, GRETIL) as fallback; Negotiate perpetual license	Legal + Data Lead

3.2.4 Risk Contingency Reserve

Table 3.9: Risk Contingency Reserve

Risk Category	Contingency Allocated (INR Lakhs)	Notes

Linguist shortage (R01)	3.00	Honorarium for 3 part-time linguists
Schedule overrun (R02, R03)	2.00	Overtime and additional resources
Infrastructure (R04)	1.00	Buffer for cloud cost overrun
Legal / Licensing (R06)	0.50	Fallback corpus licenses
Unforeseen risks	1.00	General contingency
Total Contingency Reserve	**7.50**	Within 15-20% of total budget

3.3 Experiment 3: RMMM Plan, Configuration Management & GitHub

3.3.1 RMMM (Risk Mitigation, Monitoring & Management) Plan

Monitoring Frequency

Table 3.10: RMMM Monitoring Schedule

Activity	Frequency	Responsible
Risk review meeting	Bi-weekly (alternate Wednesdays)	Project Manager
Risk register update	Weekly (every Friday)	Risk Owner
Mitigation action tracking	Weekly	Project Manager
Trigger monitoring (linguist availability)	Daily	Team Lead
Early warning indicator review	Monthly	Steering Committee

Risk Triggers and Thresholds

Table 3.11: Risk Triggers and Thresholds

Risk ID	Trigger	Response Activation Threshold
R01	Less than 2 linguists available for 2 consecutive weeks	Immediately escalate to sponsor

R02	Panini rule implementation slips by >10 days	Crash activity C (INR 25,000/day)
R03	Sandhi accuracy <80% on intermediate test	Add second linguist to validation
R04	Monthly cloud cost exceeds INR 1 Lakh	Migrate non-critical jobs to spot instances
R05	Zero paid licenses after 6 months of launch	Launch promotional pricing (50% discount)

3.3.2 Configuration Management Plan

Configuration Items (CIs)

The following items are under configuration management:

1. Panini rule base (XML files) – versioned as ruleset_v1.0, v1.1, etc.
2. Morphological analyzer source code (Python) – GitHub repository
3. Semantic parser models (PyTorch weights) – versioned with model_v1.0.pt
4. API specifications (OpenAPI YAML) – versioned with API version
5. Test datasets and gold standards – versioned with dataset_v1.0
6. Documentation (User guide, API reference) – versioned with release
7. Docker images and deployment scripts – versioned with build tag

3.3.3 Software Configuration Management: GitHub

Repository Structure

```
/sanskrit-nlp-framework/
+-- .github/
    +-- workflows/
        +-- ci.yml
        +-- deploy.yml
+-- src/
    +-- morphological_analyzer/
    +-- semantic_parser/
    +-- knowledge_extraction/
    +-- api/
+-- tests/
```

```
    +-- unit/
    +-- integration/
    +-- fixtures/
+-- data/
    +-- corpora/
    +-- rules/
    +-- gold_standards/
+-- docs/
    +-- api/
    +-- user_guide/
+-- models/
    +-- trained/
    +-- checkpoints/
+-- docker/
    +-- Dockerfile
    +-- docker-compose.yml
+-- requirements.txt
+-- README.md
+-- CHANGELOG.md
```

Branching Strategy (Git Flow)

Table 3.12: Git Flow Branching Strategy

Branch	Purpose	Protection Rules
main	Production-ready code	PR required; 2 approvals; all CI passes
develop	Integration branch	PR required; 1 approval; CI passes
feature/*	New features	No protection; deleted after merge
release/*	Release preparation	PR to main and develop
hotfix/*	Urgent production fixes	PR to main; 1 approval

CI/CD Pipeline (GitHub Actions)

Table 3.13: CI/CD Pipeline Stages

Stage	Actions	Trigger
Lint	flake8, black, isort	On push to any branch

Unit Test	pytest (minimum 80% coverage)	On push to feature/develop
Integration Test	API endpoint tests	On PR to develop
Build	Docker image build	On merge to develop
Deploy (Staging)	Deploy to staging environment	On merge to develop
Deploy (Production)	Deploy to production	On tag release/v*

3.3.4 Versioning Convention

Semantic Versioning (SemVer 2.0.0): **Major.Minor.Patch**

- **Major**: Incompatible API changes, complete Panini rule overhaul
- **Minor**: Backward-compatible functionality additions (new sutras, new API endpoints)
- **Patch**: Backward-compatible bug fixes, documentation updates

Example: `v1.0.0` – First production release with 500 Panini sutras

3.4 Experiment 4: Unit Testing with Test Cases

3.4.1 Testing Strategy Overview

Table 3.14: Testing Strategy

Test Level	Framework	Target Coverage
Unit Testing	pytest (Python)	85%
Integration Testing	pytest + FastAPI TestClient	90% of API endpoints
Regression Testing	pytest + GitHub Actions	100% of prior tests
Accuracy Validation	Custom validation suite + Linguist review	92% precision
Performance Testing	locust	< 2 sec/verse

3.4.2 Sample Test Case 1: Basic Noun (Nominative Singular)

Table 3.15: Test Case TC-MORPH-001

Test Case ID	TC-MORPH-001
Test Description	Validate morphological analysis of a simple Sanskrit noun in nominative singular
Input	raama.h (raama with visarga)
Expected Output	{word: ”raama.h”, stem: ”raama”, case: ”nominative”, number: ”singular”, gender: ”masculine”}
Actual Output	{word: ”raama.h”, stem: ”raam”, case: ”nominative”, number: ”singular”, gender: ”masculine”}
Status	FAIL
Root Cause	Sandhi rule for visarga removal incorrectly applied
Fix	Update sandhi rule: remove visarga before applying suffix stripping

3.4.3 Sample Test Case 2: Present Tense Verb

Table 3.16: Test Case TC-MORPH-015

Test Case ID	TC-MORPH-015
Test Description	Validate morphological analysis of present tense verb (3rd person singular)
Input	gacchati
Expected Output	{root: ”gam”, tense: ”present”, person: ”3rd”, number: ”singular”, voice: ”active”, pada: ”parasmaipada”}
Actual Output	{root: ”gacch”, tense: ”present”, person: ”3rd”, number: ”singular”, voice: ”active”, pada: ”parasmaipada”}
Status	FAIL
Root Cause	Irregular root transformation ”gam $\rightarrow$ gaccha” not in rule base
Fix	Add irregular verb lookup table for root ”gam” and 10 other common roots

3.4.4 Sample Test Case 3: Compound Word (Sandhi) – PASS

Table 3.17: Test Case TC-MORPH-042 (Passing)

Test Case ID	TC-MORPH-042
Test Description	Validate sandhi splitting and analysis of compound word
Input	devaalaya.h

Expected Output	{components: [”deva”, ”aalaya”], split_rule: ”guna sandhi”, analysis: [”deva: masc nom sg”, ”aalaya: masc nom sg”]}
Actual Output	{components: [”deva”, ”aalaya”], split_rule: ”guna sandhi”, analysis: [”deva: masc nom sg”, ”aalaya: masc nom sg”]}
Status	PASS

3.4.5 Test Execution Summary (Sprint 4)

Table 3.18: Test Execution Summary

Module	**Test Cases**	**Passed**	**Failed**
Morphological Analyzer	45	38	7
Semantic Parser	25	21	4
Knowledge Extraction	15	13	2
API Endpoints	12	12	0
Integration	8	8	0
Total	**105**	**92**	**13**

Overall Pass Rate: $\frac{92}{105} \times 100 = 87.6\%$

3.4.6 Defect Analysis

Table 3.19: Defect Analysis by Severity

Severity	**Description**	**Count**	**Action**
Critical	Prevents core functionality (morphological analysis)	2	Fix immediately; re-run tests
Major	Affects accuracy for specific cases	5	Fix within sprint
Minor	Edge cases or documentation issues	6	Fix in next sprint

3.4.7 Corrective Actions Based on Unit Testing

1. **Critical Defect 1** (Stemming error for visarga-ending words): Update sandhi preprocessing module – estimated 4 hours

2. **Critical Defect 2** (Root extraction for irregular verbs): Implement irregular verb table with 50 common roots – estimated 8 hours

3. **Major Defects**: Add 15 additional sandhi rules for common patterns – estimated 2 days

4. **Regression Testing**: After fixes, re-run all 105 test cases before next deployment

3.4.8 Quality Metrics Dashboard

Table 3.20: Quality Metrics Dashboard

Metric	**Target**	**Current**	**Status**
Unit Test Pass Rate	95%	87.6%	Below Target
Code Coverage	85%	79%	Below Target
Defect Density	< 2/KLOC	1.8/KLOC	On Track
Mean Time to Fix (Critical)	< 24 hrs	18 hrs	On Track
Linguist Validation Accuracy	92%	89%	Below Target

3.4.9 Conclusion for Unit 3

The project is currently slightly behind schedule (SPI = 0.92) and over budget (CPI = 0.87) at Month 6. Earned Value Analysis indicates an EAC of INR 42.53 Lakhs, requiring corrective actions [2]. Corrective actions including additional NLP engineer and re-allocated contingency are expected to improve performance to SPI = 0.97 and CPI = 0.94. Risk management has identified six key risks with mitigation strategies and a contingency reserve of INR 7.5 Lakhs. Configuration management is fully implemented using GitHub with Git Flow branching and CI/CD pipelines. Unit testing shows 87.6% pass rate with two critical defects being addressed immediately. Quality metrics indicate need for improved test coverage and linguist validation accuracy before production release [3].